

Restaurant Management System – User Manual

Project Title: Restaurant Management System.

Course: CSC 478 – Software Engineering Capstone

Team Name: Error 404

Team Members: Haripriya Koneru, Krishna Kumar Nimmagadda, Venkata Sai Gowtham Panyam, Vansh Parihar, Brijeshkumar Mansukhbhai Raiyani, Pratham Patel, Vijaya Venkateswara Rao Rowthu

Document: User Manual

Frontend Repository: https://github.com/Patel-Pratham-3492/Error_404

Backend Repository: https://github.com/Patel-Pratham-3492/Error_404_server

Frontend dev server default: `http://localhost:5173`

Backend dev server default: `http://localhost:5000`

The Restaurant Management System (RMS) is a web-based application designed to streamline daily restaurant operations by providing role-specific tools for hosts, waiters, chefs, managers, and owners. The system replaces manual, paper-based workflows with a digital platform that improves accuracy, communication, and efficiency across the front-of-house and back-of-house teams.

This User Manual provides clear instructions for installing, configuring, and running both the **frontend (React + Vite)** and **backend (Node.js + Express + MongoDB)** components of the RMS. It is intended for developers, testers, instructors, and restaurant staff who need to operate or evaluate the system.

The document explains:

- Required software and environment setup
- How to install and run the frontend and backend
- Basic configuration steps using environment variables
- How to log in and use role-based dashboards

By following this guide, users will be able to fully deploy the system locally, interact with all major features, and understand how the RMS supports the restaurant's workflow—from table assignment to order processing, food preparation, menu management, staff controls, and reporting.

1. Prerequisites

Before installing and running the project locally, ensure you have:

- Node.js (v14+ recommended) and npm installed. Verify with:
 - `node -v`
 - `npm -v`
- MongoDB running locally **or** a MongoDB Atlas connection string.
- Git (to clone the repositories).

2. Clone the Repositories

Open a terminal and run:

```
# Frontend
git clone https://github.com/Patel-Pratham-3492/Error_404.git

# Backend
git clone https://github.com/Patel-Pratham-3492/Error_404_server.git
```

Change into each repo to run the install steps below.

3. Backend Installation & Run (Node.js + Express + MongoDB)

3.1 Install dependencies

In the backend project folder:

```
cd Error_404_server
npm install
```

3.2 Configure environment variables

Create a `.env` file at the project root (same level as `index.js` or `server.js`) with the variables below (example):

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/rms_db
SESSION_SECRET=yourStrongSessionSecretHere
COOKIE_SECURE=false           # true if using HTTPS in production
NODE_ENV=development
FRONTEND_URL=http://localhost:5173  # for CORS and cookie settings
```

- If using MongoDB Atlas, set `MONGO_URI` to the Atlas connection string.
- `SESSION_SECRET` should be a long random string in production.

3.3 Start the backend

Run (simple run):

```
node index.js
```

Or if a start script exists in `package.json`:

```
npm start
```

You should see server logs and a message like `Server listening on port 5000`. The backend will be available at:

```
http://localhost:5000
```

4. Frontend Installation & Run (React + Vite)

4.1 Install dependencies

In the frontend project folder:

```
cd Error_404
npm install
```

4.2 Configure environment variables (if any)

If the frontend uses an `.env` file (Vite), create `.env` or `.env.development` with the backend base URL:

```
VITE_API_BASE_URL=http://localhost:5000/api
```

(If your frontend reads a different variable name, use that. Otherwise the default axios base URL may be defined in `src/services/api.js`.)

4.3 Run the dev server

```
npm run dev
```

Vite's dev server will start and usually show:

```
Local: http://localhost:5173
```

Open that URL in Chrome or Edge.

5. Usage Instructions

1. Open frontend: `http://localhost:5173`
2. Login using test credentials (seeded or created). Login calls `POST /api/login` and receives a session cookie.
3. After login, you'll be redirected to the dashboard according to your role:
 - **Host** — table management: view open/occupied tables, assign tables, mark occupied/open.
 - **Waiter** — create/modify orders, view assigned tables, payment modal.
 - **Chef** — view incoming orders, mark items in-progress / done (if implemented).
 - **Manager** — menu CRUD (create/update/delete), staff management, assign tables.
 - **Owner** — access reports and owner-only functions.
4. Typical actions:
 - **Manager**: Manage menu → add items, change price, toggle availability.
 - **Waiter**: Select table → Add items → Submit order → Open payment when done.
 - **Chef**: Monitor orders queue → Click Preparing/Done on items (where supported).
 - **Host**: Assign and free tables.